

1. Автентифікація та Користувачі (Authentication & Users)

Цей блок відповідає за безпеку системи, вхід працівників та управління їхніми ролями (Admin / Teacher).

- **POST /api/v1/auth/login/**
 - **Логіка:** Приймає номер телефону та пароль. Перевіряє їх у базі даних і, якщо все вірно, генерує та повертає JWT-токен доступу.
 - **Призначення:** Використовується на головному екрані входу в систему.
- **GET /api/v1/users/me/**
 - **Логіка:** Зчитує токен із запиту та повертає дані того користувача, який зараз залогінений (його ім'я, роль та до яких філій він має доступ).
 - **Призначення:** Фронтенд використовує це, щоб зрозуміти, які кнопки ховати. Наприклад, якщо роль **teacher**, фронтенд приховує кнопку "Створити філію".
- **POST /api/v1/users/**
 - **Логіка:** Створює нового співробітника системи. Доступно лише для адміністратора.
 - **Призначення:** Форма додавання нового вчителя або менеджера у систему.

2. Філії (Branches)

Управління навчальними центрами. Філія є кореневою сутністю — усі інші дані (учні, предмети) прив'язуються до неї.

- **GET /api/v1/branches/**
 - **Логіка:** Повертає масив усіх активних філій, до яких має доступ поточний користувач.
 - **Призначення:** Використовується для відображення списку локацій на головному дашборді або у випадяючому списку при створенні уроку.
- **POST /api/v1/branches/**
 - **Логіка:** Приймає назву, адресу та місто. Створює новий запис у таблиці **Branch**.
 - **Призначення:** Форма "Додати нову філію".
- **PATCH /api/v1/branches/{id}/status/**
 - **Логіка:** Змінює статус філії з **active** на **archived**. М'яке видалення (soft delete), щоб не втратити історичні дані про розклади цієї філії.

3. Предмети (Subjects)

Дисципліни, які викладаються в конкретній філії.

- **GET /api/v1/branches/{branch_id}/subjects/**
 - **Логіка:** Шукає та повертає список усіх предметів, які належать конкретній філії (за `branch_id`).
 - **Призначення:** Використовується при створенні групи або призначенні тарифу, щоб адмін міг обрати предмет лише з тих, що доступні в цій філії.
- **POST /api/v1/branches/{branch_id}/subjects/**
 - **Логіка:** Створює новий предмет (наприклад, "Python Basic") та автоматично прив'язує його до вказаної філії.

4. Студенти (Students)

Управління базою клієнтів (учнів).

- **GET /api/v1/branches/{branch_id}/students/**
 - **Логіка:** Повертає список усіх учнів, зареєстрованих у конкретній філії. Підтримує пагінацію (посторінковий вивід), якщо учнів багато.
 - **Призначення:** Таблиця учнів у панелі адміністратора.
- **POST /api/v1/branches/{branch_id}/students/**
 - **Логіка:** Приймає персональні дані (ПІБ, контакти, інфо про батьків) і створює картку учня, прив'язуючи її до філії.
 - **Призначення:** Форма реєстрації нового клієнта.
- **GET /api/v1/students/{id}/**
 - **Логіка:** Повертає всю детальну інформацію про одного конкретного учня за його ID.
 - **Призначення:** Сторінка профілю учня (де також будуть виводитися його тарифи та групи).

5. Групи (Groups)

Об'єднання студентів для спільних занять.

- **POST /api/v1/branches/{branch_id}/groups/**
 - **Логіка:** Створює нову групу в межах філії.
- **POST /api/v1/groups/{id}/students/**
 - **Логіка:** Приймає масив ID студентів і створює записи у проміжній таблиці `Student_groups`, додаючи учнів до цієї групи.
 - **Призначення:** Інтерфейс комплектування групи перед стартом навчання.

6. Тарифи та Підписки (Subscriptions)

Фінансова логіка системи.

- **GET /api/v1/branches/{branch_id}/plans/**
 - **Логіка:** Повертає список доступних тарифних планів (цінових сіток) для філії.

- **POST /api/v1/students/{student_id}/subscriptions/**
 - **Логіка:** Створює запис у таблиці `Student_Subscriptions`. Приймає `plan_id`, `subject_id` та `start_date`. Фіксує, за яким тарифом учень буде оплачувати конкретний предмет.
 - **Призначення:** Кнопка "Призначити тариф" у профілі студента.

7. Розклад та Уроки (Schedules & Lessons)

Головне ядро бізнес-логіки для уникнення накладок.

- **POST /api/v1/lesson-templates/**
 - **Логіка:** Створює правило регулярних занять. Приймає дні тижня, час, вчителя, предмет і групу/студента. Система на основі цього шаблону автоматично генерує записи в таблиці `Lessons` на період дії шаблону.
 - **Призначення:** Автоматизація роботи менеджера — замість створення 10 уроків вручну, створюється один шаблон.
- **GET /api/v1/lessons/**
 - **Логіка:** Повертає список згенерованих уроків. Обов'язково приймає параметри фільтрації дат (`date_from`, `date_to`) та вчителя.
 - **Призначення:** Відображення календаря занять для адміністратора (усі вчителі) та для конкретного вчителя (тільки його уроки).

8. Відвідуваність (Attendance)

Контроль проведення занять.

- **POST /api/v1/lessons/{lesson_id}/attendance/**
 - **Логіка:** Приймає масив статусів (присутній/відсутній) для кожного учня на конкретному уроці та зберігає їх у таблицю `Attendances`. Після цього статус самого уроку змінюється на `COMPLETED`.
 - **Призначення:** Інтерфейс вчителя в кінці заняття для відмітки журналу.

Усі маршрути містять префікс `v1` (Version 1). Це стандартна практика (Best Practice) у розробці програмного забезпечення, яка гарантує безпечне масштабування системи в майбутньому. Якщо в подальшому бізнес-логіка докорінно зміниться (наприклад, зміняться обов'язкові поля для реєстрації), розробники створять маршрути `/api/v2/`. Це дозволить новим версіям клієнтських додатків працювати з новим функціоналом, тоді як старі додатки (наприклад, не оновлені мобільні застосунки клієнтів) продовжуватимуть стабільно і без помилок працювати зі старими маршрутами `v1`.

